Blazor: lista prodotti, pulsanti e pagina di dettaglio

Lezione 3 — 3IOT, SST-ICT-01 — Rotte con parametri, render mode interattivo, NavMenu, pulsanti con @onclick, NavigationManager, [Parameter]; compito: catalogo da database

14/10/2026

1 Da dove ripartiamo: link statici e link dinamici

Nota

Ripasso di quanto visto in A02, ripreso alla lavagna perché è la base di tutta la lezione di oggi.

- Con le tecnologie più vecchie (HTML statico, PHP) il link punta a un **file**: `www.ial.it/corsi.html`. Se il file viene rinominato, i link salvati dai motori di ricerca o da altri siti **smettono di funzionare**.
- Con le tecnologie più recenti il link contiene un **descrittore**, una parola chiave: `www.ial.it/corsi`. Il file che risponde può chiamarsi in qualunque modo (per esempio `CorsiAttivi.razor`): è la **direttiva `@page`** a dire a quali indirizzi risponde.
- Una pagina può rispondere a **più indirizzi**: basta mettere più righe `@page`. Così chi arriva da un link vecchio o diverso (`/corsiavviati`) finisce comunque nella pagina giusta.

Passare informazioni alla pagina

Definizione

Il vecchio modo (GET). Le variabili si accodano all'indirizzo dopo un **punto di domanda**, separate dalla **e commerciale**:

```
www.ial.it/corsi.html?nome=Mario&cognome=Rossi
```

Il modo di Blazor: parametri di rotta. I valori diventano **pezzi dell'indirizzo**: `www.ial.it/corsi/Mario/Rossi`. Nella pagina si dichiara la rotta con i nomi dei parametri tra **graffe**, e nel blocco `@code` una proprietà per ciascuno, marcata con **`[Parameter]`**.

```razor
@* CorsiAttivi.razor *@
@page "/corsi"
@page "/corsiavviati"
@page "/corsi/{Nome}/{Cognome}"

@code {
    [Parameter]
    public string? Nome { get; set; }

    [Parameter]
    public string? Cognome { get; set; }
}
```

- Le tre righe `@page` fanno rispondere la pagina sia senza parametri sia con nome e cognome. Ogni combinazione che si vuole gestire va dichiarata.
- Quando la pagina viene chiamata, la prima cosa che fa è **valorizzare** le proprietà `[Parameter]` con i pezzi dell'indirizzo. Poi si possono usare per filtrare o estrarre dati.
- Il **punto di domanda** dopo `string` dice che il valore **può mancare** (essere `null`): la pagina non sa in anticipo quale nome arriverà, o se arriverà. Senza il punto di domanda funziona lo stesso, ma Visual Studio segnala un avviso.

Attenzione

Nota di correzione. Alla lavagna i parametri erano scritti in forma breve, come campi (`public string? Nome;`). In Blazor `[Parameter]` va messo su una **proprietà**, con `{ get; set; }`, come nel listato: è anche l'errore che è emerso nel laboratorio (§5.8).

2 L'obiettivo: elenco e dettaglio

L'obiettivo di questo blocco di lezioni è un'applicazione web, già mostrata come esempio finito, che:

- apre una pagina con l'**elenco di tutti i prodotti** (al caricamento esegue una **SELECT** di tutti i prodotti dell'archivio);
- per ogni prodotto ha un **pulsante** che apre la **pagina di dettaglio** di quel prodotto;
- la pagina di dettaglio riceve nell'indirizzo l'**Id del prodotto** e con quello recupera i dati (`SELECT ... WHERE Id = ...`).

Serve una **classe** che descrive il prodotto (Id, nome, descrizione, prezzo, data di scadenza, ...), in un file `.cs` in una cartella del progetto (nell'esempio `Models/Prodotti.cs`). Oggi il collegamento al database non è stato fatto in aula: l'elenco è una **lista in memoria**, e il database diventa il compito (§7).

3 Pagine statiche e pagine interattive: `@rendermode`

Definizione

- Una pagina **Razor statica** viene costruita dal **server** al momento della richiesta e spedita al browser già pronta. Per vedere dati nuovi bisogna **richiedere di nuovo** la pagina (aggiornare).
- Una pagina **Blazor interattiva** tiene **aperto un canale** di comunicazione tra il browser e il server (tecnologia **SignalR**): le modifiche arrivano **in tempo reale**, senza aggiornare la pagina, e i pulsanti possono chiamare codice C# sul server.

Noi lavoriamo con le pagine Blazor interattive. Bisogna dirlo al progetto, altrimenti i pulsanti con `@onclick` **non fanno niente** (si potrebbe ripiegare su una `form`, ma è proprio il tipo di chiamata che vogliamo eliminare).

Esempio

In aula: in fondo al file **`Routes.razor`** si aggiunge una riga, così vale per tutte le pagine (lo si scrive una volta sola, ma **in ogni progetto nuovo**):

```
<Router AppAssembly="typeof(Program).Assembly" NotFoundPage="typeof(Pages.NotFound)">
    <Found Context="routeData">
        <RouteView RouteData="routeData" DefaultLayout="typeof(Layout.MainLayout)" />
        <FocusOnNavigate RouteData="routeData" Selector="h1" />
    </Found>
</Router>

@rendermode InteractiveServer
```

`@rendermode` tutto minuscolo, `InteractiveServer` con la **I** e la **S** maiuscole. Se Visual Studio non la colora, probabilmente c'è un errore di scrittura.

Nota

Precisazione. In aula si è detto che la riga si può mettere anche in `App.razor` o in `_Imports.razor`; il docente consiglia `Routes.razor`, ed è la forma che abbiamo provato e che funziona. Quando, creando il progetto, si sceglie l'interattività *globale*, Visual Studio scrive invece il render mode in `App.razor`, sui componenti: `<Routes @rendermode="InteractiveServer" />` (e lo stesso su `HeadOutlet`). Il nome `InteractiveServer` si può scrivere così, senza prefisso, perché il modello del progetto contiene già in `_Imports.razor` la riga `@using static Microsoft.AspNetCore.Components.Web.RenderMode`.

4 Link di navigazione e pulsanti

Sono due cose diverse:

- i **link di navigazione** a bordo pagina (il menu a sinistra) portano a pagine **generali**, sempre uguali;
- un **pulsante accanto a ogni prodotto** apre la **stessa pagina di dettaglio**, ma deve dire

quale prodotto: al pulsante del prodotto 1 si passa l'Id 1, a quello del prodotto 2 l'Id 2, e così via.

Definizione

Lo schema del pulsante (alla lavagna).

```
<button @onclick="() => nomeMetodo(id)">
    Testo del pulsante
</button>
```

- **`@onclick`** dice che cosa fare quando si clicca.
- **`() =>`** (tonde vuote, uguale, maggiore: una **freccia**) introduce il codice da eseguire; serve per poter passare un valore al metodo.
- Tra le tonde del metodo si mette il dato da passare, qui l'Id del prodotto.

L'aspetto (colori, dimensioni) dipende dalla **classe** CSS: con **Bootstrap** (sito `getbootstrap.com`) si usano le classi già pronte, per esempio `btn btn-primary`.

5 Laboratorio guidato: il progetto Lezione3

5.1 Progetto nuovo e prima prova

1. In Visual Studio: nuovo progetto **App Web Blazor** (nome in aula: `Lezione3`, ognuno sceglie il suo).
2. In `Routes.razor` si aggiunge `@rendermode InteractiveServer` e si avvia per controllare che non ci siano errori.
3. L'applicazione gira su **localhost**, cioè in locale; a sinistra ci sono già tre voci di menu (Home, Counter, Weather) che **non tocchiamo**.

5.2 Una pagina nuova con più indirizzi

1. Cartella `Components/Pages`, tasto destro, **Aggiungi** → **Componente Razor**.
2. Il nome del file deve avere **la prima lettera maiuscola** (`Listaprodotti.razor`): lo richiede il sistema, e tanti sbagliano proprio qui.
3. In cima si scrivono le rotte. Per far vedere che si può, la pagina risponde a tre indirizzi:

```
@page "/listaprodotti"
@page "/tutti-i-prodotti"
@page "/prodotti"

<h3>Listaprodotti</h3>

<p>Pagina dei prodotti appena creata</p>
```

Avviando l'applicazione, alla pagina si arriva **solo scrivendo l'indirizzo** nella barra del browser: `/listaprodotti`, `/prodotti` e `/tutti-i-prodotti` portano tutti alla stessa pagina.

5.3 La voce di menu

L'utente non deve ricordare a memoria gli indirizzi: gli si dà una voce di menu. Il menu è nel file **`Components/Layout/NavMenu.razor`**: contiene tre blocchi quasi uguali, uno per voce. Il `div` serve a impaginare; il **`NavLink`** è il link vero e proprio. Si copia il terzo blocco, lo si incolla dopo e si cambiano **`href`** (l'indirizzo, relativo al sito) e il **testo**:

```
<div class="nav-item px-3">
    <NavLink class="nav-link" href="/tutti-i-prodotti" Match="NavLinkMatch.All">
        <span class="bi bi-list-nested-nav-menu" aria-hidden="true"></span> Lista Prodotti
    </NavLink>
</div>
```

Nota

L'icona. La classe nello `span` sceglie l'icona; l'elenco delle icone è sul sito di Bootstrap, alla voce *Icons*. Nel modello del progetto le icone del menu sono definite nel file `NavMenu.razor.css`: un nome nuovo non compare se non è definito lì. In aula ci si è fermati qui: la parte grafica è libera, quello che conta sono **il link e il nome** della voce.

5.4 La classe Prodotto e la lista

Nel blocco `@code` della pagina si definiscono la **classe** e una **lista** di tre prodotti; nella parte HTML un **`@foreach`** stampa nome e prezzo di ciascuno.

```
@foreach (var prodotto in Prodotti)
{
    <div>
        <h4>@prodotto.Nome</h4>
        <p>Prezzo: @prodotto.Prezzo.ToString("C")</p>
    </div>
}

@code {
    public class Prodotto
    {
        public int Id { get; set; }
        public string Nome { get; set; }
        public decimal Prezzo { get; set; }
    }

    public List<Prodotto> Prodotti = new List<Prodotto>()
    {
        new Prodotto { Id = 1, Nome = "Pane", Prezzo = 10.99m },
        new Prodotto { Id = 2, Nome = "Latte", Prezzo = 5.49m },
        new Prodotto { Id = 3, Nome = "Uova", Prezzo = 3.99m }
    };
}
```

- `ToString("C")` formatta il prezzo come **valuta** (in aula: `10,99 €`). La `m` dopo i numeri indica un valore `decimal`.
- Il `@foreach` va nella parte **front end**, cioè nell'HTML, **fuori** dal blocco `@code`.

Attenzione

Errori incontrati in aula.

- "La proprietà ... deve avere almeno una funzione di accesso": mancava `{ get; set; }` su una proprietà.
- Punto e virgola mancante dopo la chiusura della lista (`};`).
- La lista scritta **dentro** la classe invece che sotto, fuori dalla sua graffa: con un'**indentazione** ordinata si vede subito dove finisce ogni blocco.
- Se la compilazione fallisce e si avvia lo stesso, Visual Studio può eseguire la **versione precedente**: si sta provando codice vecchio. Prima si eliminano gli errori.

5.5 Il pulsante e NavigationManager

Accanto a ogni prodotto si aggiunge un pulsante (classe `btn btn-primary`, blu) che chiama un metodo. Per spostarsi su un'altra pagina da codice si usa **`NavigationManager`**, che va "tirato dentro" in cima alla pagina con **`@inject`**.

```
@inject NavigationManager NavigationManager
...
        <button class="btn btn-primary" @onclick="apriPagina">
            Apri Dettaglio
        </button>
...
@code {
    public void apriPagina()
    {
        NavigationManager.NavigateTo("/dettaglio-prodotto");
    }
}
```

Attenzione

Tonde sì o no? Scrivendo `@onclick="apriPagina()"`, con le tonde, il metodo verrebbe **eseguito subito** e a `@onclick` arriverebbe il suo risultato, che è `void`: errore "impossibile convertire da `void` a `EventCallback`". Senza argomenti si scrive solo il nome (`apriPagina`); con argomenti si usa la freccia (§5.7).

Ora i pulsanti ci sono, ma restano tre problemi:

1. la pagina `/dettaglio-prodotto` **non esiste**;
2. qualunque pulsante si prema, si va **sempre allo stesso indirizzo**;
3. la classe e la lista dei prodotti esistono **solo** nella pagina dell'elenco, e servirebbero anche nel dettaglio.

5.6 La pagina di dettaglio

Come prima: `Components/Pages`, **Aggiungi componente Razor**, nome con la maiuscola (in aula `Dettaglio.razor`). Il nome del file è libero; conta che la rotta nella `@page` sia **coerente** con l'indirizzo

usato in `NavigateTo`.

5.7 Passare l'Id del prodotto

1. Nel pulsante si passa l'Id con la freccia: `() => apriPagina(prodotto.Id)`. `prodotto` è la variabile del `@foreach`: a ogni giro vale il pane, poi il latte, poi le uova. Funziona come il **passaggio di parametri per valore** alle funzioni.
2. Il metodo deve **ricevere** il dato: `apriPagina(int id)`.
3. L'Id entra nell'indirizzo con una **stringa interpolata**: il **dollaro** davanti alle virgolette permette di inserire una variabile tra graffe.

```
<button class="btn btn-primary" @onclick="() => apriPagina(prodotto.Id)">
    Apri Dettaglio
</button>
...
public void apriPagina(int id)
{
    NavigationManager.NavigateTo($"/dettaglio-prodotto/{id}");
}
```

Ora ogni pulsante chiama un indirizzo diverso: `/dettaglio-prodotto/1`, `/dettaglio-prodotto/2`, `/dettaglio-prodotto/3`.

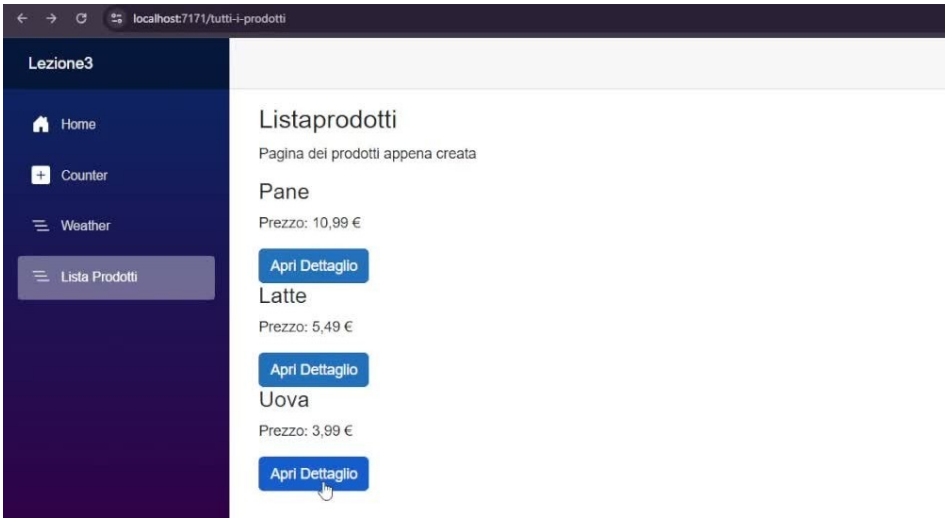


Figura 1: L'applicazione Lezione3 in aula: la voce di menu "Lista Prodotti" porta a `/tutti-i-prodotti`; ogni prodotto ha il suo pulsante "Apri Dettaglio".

5.8 Gli errori sul parametro `id`

Attenzione

Primo errore. Cliccando un pulsante compare in fondo alla pagina un avviso di errore non gestito; ricaricando, il dettaglio dice: *InvalidOperationException: Object of type 'Lezione3.Components.Pages.Dettaglio' does not have a property matching the name 'id'*. La pagina di dettaglio riceve un `id` dall'indirizzo, ma **non ha una proprietà** dove metterlo. Manca il `[Parameter]`.

Attenzione

Secondo errore. Aggiunto `[Parameter] public int id { get; set; }`, l'errore cambia: *Unable to cast object of type 'System.String' to type 'System.Int32'*. Il pezzo di indirizzo arriva come **testo**: la proprietà deve essere `string` (l'autocompletamento aveva proposto `int`, ed era sbagliato).

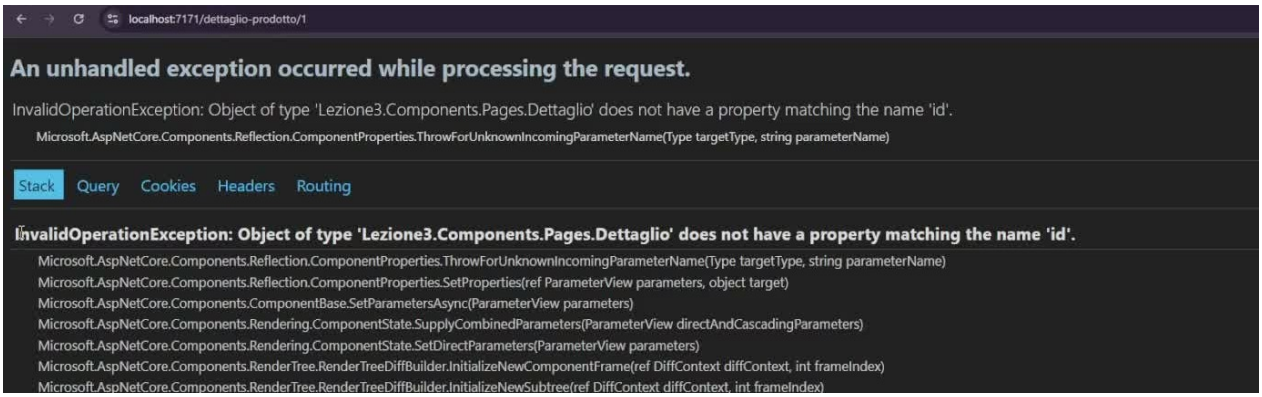


Figura 2: Il primo errore: la pagina di dettaglio non ha una proprietà che corrisponda al parametro `id` della rotta.

```
@page "/dettaglio-prodotto/{id}"

<h3>Dettaglio</h3>

<p>Pagina di dettaglio del prodotto</p>

@code {
    [Parameter]
    public string? id { get; set; }
}
```

Nota

Precisazione (non fatta in aula). Se si vuole l'Id già come numero si può aggiungere un **vincolo** alla rotta: `@page "/dettaglio-prodotto/{id:int}"` e poi `public int id { get; set; }`. Con il vincolo, un indirizzo con un Id non numerico non trova la pagina.

Esempio

Quattro insegnamenti, secondo il docente.

1. Nemmeno il docente sa tutto.
2. L'**autocompletamento** non sempre fa quello che deve.
3. Gli errori in genere vengono segnalati, ma bisogna **leggerli**: altrimenti la soluzione non si trova.
4. I messaggi sono in **inglese**: serve saperlo leggere.

6 Dati duplicati: perché serve il database

Per far funzionare il dettaglio, in aula classe e lista sono state **copiate** anche in `Dettaglio.razor`. Il docente lo dice chiaramente: è una **porcata**. Duplicare dati su pagine diverse **non si fa**: qui i prodotti sono tre, ma un negozio può averne 7000, e non si porta una lista di 7000 prodotti copiata in ogni pagina. I dati vanno in un **database**: ogni pagina chiede quello che le serve (l'elenco fa la SELECT di tutti, il dettaglio quella con l'Id ricevuto). La pagina di dettaglio in aula riceve l'Id ma **non mostra ancora** i dati del prodotto: è il passo del compito.

7 Il compito: catalogo da database

Attenzione

Consegna: entro le ore 20 del 27 ottobre, su Classroom. Si consegna un file **ZIP** con tutto il progetto Visual Studio e il **database** (file `.mdf`) messo in una cartella `database` dentro la cartella del progetto. Non servono le librerie.

- Un **catalogo di 100 prodotti** memorizzati in un **database**. Il tipo di prodotti è libero (alimentari, videogiochi, una pizzeria…). Per inventarli e inserirli si possono anche usare strumenti di generazione automatica: come si popola un database lo sapete già.
- Una **pagina di elenco** con tutti i prodotti, presentati come preferite: box, tabella, testo. La grafica è libera.
- Per ogni prodotto un **pulsante** che apre la **pagina di dettaglio**, con informazioni in più. Esempio della pizzeria: nell'elenco "Margherita, 7,50 €"; nel dettaglio anche gli ingredienti e una foto, con il tag `<img src="...">`.
- In aula si è parlato anche di **aggiungere un nuovo prodotto** (un pulsante nell'elenco o una pagina apposita) e di modificarlo dal dettaglio: *da confermare con il docente* quali di queste parti sono richieste.

Nota

Sull'uso dell'intelligenza artificiale. Si può usare, ma il voto deve corrispondere a quello che si sa fare: all'esame non ci sarà, e una A in programmazione con il foglio bianco all'esame non porta da nessuna parte. L'obiettivo è chiudere questa parte per dedicarsi poi alla **comunicazione con il PLC**.

8 Cosa ricordare di questa lezione

- Link **dinamici**: la pagina risponde agli indirizzi dichiarati con **`@page`**, anche più d'uno.
- Parametri nell'indirizzo: `{Nome}` nella rotta + **`[Parameter]`** su una **proprietà** con lo stesso nome.
- **`@rendermode InteractiveServer`** (in aula in `Routes.razor`) rende le pagine interattive: senza, `@onclick` non funziona.
- Nome del file del componente con la **maiuscola**; voce di menu con un **`NavLink`** in `NavMenu.razor`.

- Pulsante con Id: `@onclick="() => apriPagina(prodotto.Id)"` e metodo con `int id`.
- Navigazione da codice: `@inject NavigationManager`, poi `NavigateTo` con l'indirizzo costruito con `$"...{id}"`.
- Il parametro di rotta arriva come **testo**: `string?`, oppure vincolo `{id:int}`.
- Leggere sempre il **messaggio d'errore**.
- Dati duplicati tra pagine: **no**, vanno nel **database**. Compito entro il **27/10 alle 20**.